

+ 26

Writing High Performance Parsers Using State Machines

TORBEN THAYSEN



20
26



September 12 - 18

Parsers

- Specific context: programming languages
- Transforms source code into AST
- Done in two steps, lexing then parsing
- Used by: compilers, language servers, formatters, ...

Motivation

- Parser and lexer for [Torbenx/charge](#)
- [Carbon Compiler Design](#) at C++Now 2023
 - Challenge: Parse and lex at 10M lines/sec
 - Combined at 5M lines/sec
- [Reported hit](#) at NDC Toronto 2026 on Apple M4
 - Lex at 12M lines/sec
 - Parse+Lex at 7M lines/sec

Lexing

- Breaks input into tokens
 - Identifiers
 - Operators
 - Literals
- Context independent
- Alphabet for parsers

```
conference = "CppCon" + to_string(2026)
```

Lexing

- Breaks input into tokens
 - Identifiers
 - Operators
 - Literals
- Context independent
- Alphabet for parsers

conference = "CppCon" + to_string(2026)

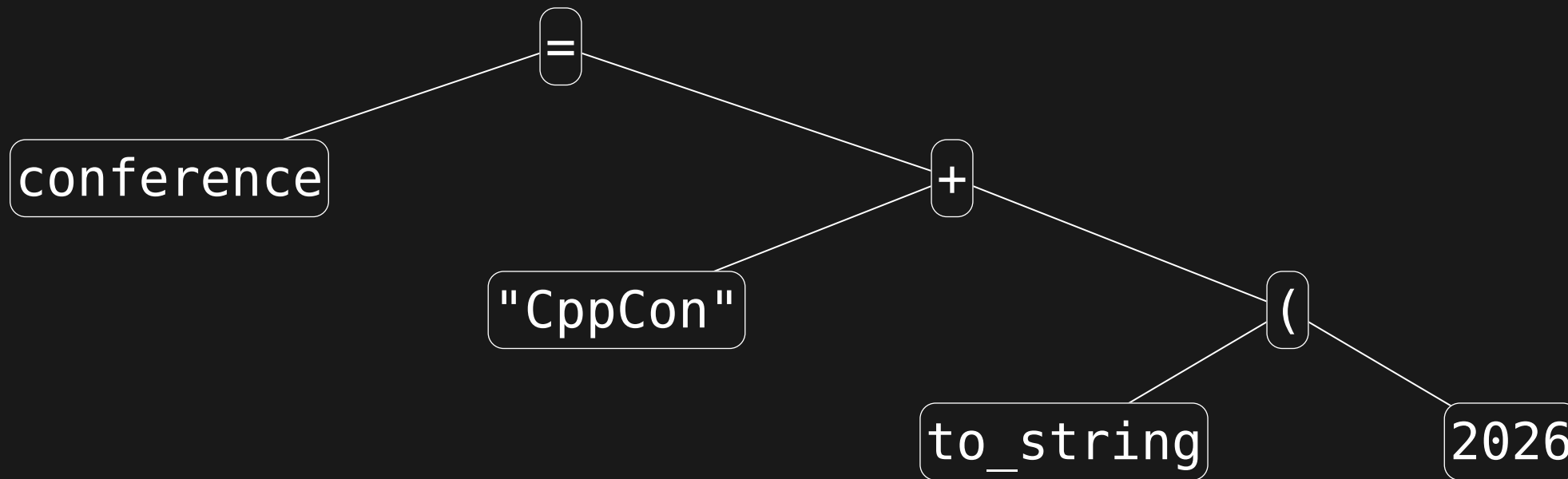
Parsing

- Consumes a token stream
- Validates syntax against grammar
- Recognizes contextual meaning of tokens
- Outputs a parse tree or AST

conference = "CppCon" + to_string(2026)

Parsing

- Consumes a token stream
- Validates syntax against grammar
- Recognizes contextual meaning of tokens
- Outputs a parse tree or AST



Main Loop

```
1  for (;;) {  
2      sourcePosition = skipWhitespace(sourcePosition);  
3      Token tok = Token::Invalid;  
4      switch (sourcePosition[0]) {  
5          case ',':  
6              tok = Token::Comma;  
7              sourcePosition += 1;  
8              break;  
9          ...  
10     }  
11     output.emitToken(tok, /* source-information */);  
12 }
```

+ Tokens

```
1 case '+':  
2     // Tokens: ++ += +  
3     if (sourcePosition[1] == '+') {  
4         tok = Token::PlusPlus;  
5         sourcePosition += 2;  
6     } else if (sourcePosition[1] == '=') {  
7         tok = Token::PlusEqual;  
8         sourcePosition += 2;  
9     } else {  
10        tok = Token::Plus;  
11        sourcePosition += 1;  
12    }  
13    break;
```

+ Tokens

```
0  switch (sourcePosition[0]) {  
1  case '+':  
2      // Tokens: ++ += +  
3      if (sourcePosition[1] == '+') {  
4          tok = Token::PlusPlus;  
5          sourcePosition += 2;  
6      } else if (sourcePosition[1] == '=') {  
7          tok = Token::PlusEqual;  
8          sourcePosition += 2;  
9      } else {  
10         tok = Token::Plus;  
11         sourcePosition += 1;  
12     }  
13     break;  
14 }  
15 output.emitToken(tok, /* source-information */);
```

String Literals

```
1 case '':  
2     tok = Token::StringLiteral;  
3     sourcePosition = skipStringLiteral(sourcePosition);  
4     sourcePosition += 1;  
5     break;
```

Note: No processing / unescaping

/ Tokens And Comments

```
1 case '/':
2     if (sourcePosition[1] == '*') {
3         sourcePosition = skipBlockComment(sourcePosition + 2);
4         sourcePosition += 2;
5         output.recordComment(CommentKind::Block, /* source-information */);
6         continue;
7     } else if (sourcePosition[1] == '/') {
8         sourcePosition = skipToEndOfLine(sourcePosition + 2);
9         output.recordComment(CommentKind::Line, /* source-information */);
10        continue;
11    } else if (sourcePosition[1] == '=') {
12        tok = Token::SlashEqual;
13        sourcePosition += 2;
14    } else {
15        tok = Token::Slash;
16        sourcePosition += 1;
17    }
18    break;
```

Identifiers

- Key decision: identifier table or not
- Extremely useful for compilers
- Very hash table heavy
 - [Matt Kulukundis at CppCon 2017](#)
 - [Malte Skarupke at C++Now 2018](#)
- Focus on formatters
 - Need keyword recognition only

Identifiers

```
1 case 'A'..'Z':  
2 case 'a'..'z':  
3 case '_':  
4     const char* begin = sourcePosition;  
5     sourcePosition = skipIdentifier(sourcePosition + 1);  
6     auto* entry = KeywordTable::get(begin, sourcePosition);  
7     tok = entry ? entry->token : Token::Identifier;  
8     break;
```

KeywordTable generated by gperf

Switch And Branch Summary

- Branches for:

```
( ) [ ] { } ~ . , : ; + - * / % ^ ! & | < > =  
:: ++ -- && || << >> ==  
+= -= *= /= %= ^= != &= |= <= >=  
&&= ||= <<= >>=  
-> =>  
// /*
```

- Integer, string and character literals
- Identifiers / keywords
- New lines and null terminator
- 30 switch cases and 58 total branches

Benchmark Setup

```
1  const char* lexSwitchAndBranch(const char* sourcePosition, LexerOutput& output);
2
3  void runGoogleBenchmark(benchmark::State& state, std::string file) {
4      std::string source = readFile(file);
5
6      for (auto _ : state) {
7          LexerOutput output { source };
8          const char* end = lexSwitchAndBranch(source.data(), output);
9          assert(end == source.data() + source.size());
10     }
11 }
```

Benchmark Setup

```
1  const char* lexSwitchAndBranch(const char* sourcePosition, LexerOutput& output);
2
3  void runGoogleBenchmark(benchmark::State& state, std::string file) {
4      std::string source = readFile(file);
5
6      for (auto _ : state) {
7          LexerOutput output { source };
8          const char* end = lexSwitchAndBranch(source.data(), output);
9          assert(end == source.data() + source.size());
10     }
11
12     LexerOutput output { source };
13     lexSwitchAndBranch(source.data(), output);
14     auto bytes = source.size();
15     auto lines = output.lines.size();
16     auto tokens = output.tokens.size();
17
18     state.counters["bytes"] = benchmark::Counter(
19         bytes, benchmark::Counter::kIsIterationInvariantRate);
20     state.counters["lines"] = benchmark::Counter(
21         lines, benchmark::Counter::kIsIterationInvariantRate);
22     state.counters["tokens"] = benchmark::Counter(
```

Switch And Branch Benchmark

- 50000 lines of AI-generated input data
- About 15.5M lines/sec
- At 220 (user space) cycles/line
 - Doesn't measure syscalls
 - Doesn't measure custom allocator

What To Optimize

- `skipIdentifier` and friends $\longrightarrow$ SIMD
- Identifier table $\longrightarrow$ hash table
- Token recognition $\longrightarrow$ branch heavy
 - `token-begin` $\mapsto$ (`token-enum`, `advance`)

How To Reduce Branches

- Completely branchless is impractical
 - Many different `skip` functions
- Reduce as much as possible
 - One case per `skip` function
 - Plus one case for punctuations
- Need to recognize all of:

```
( ) [ ] { } ~ . , : ; + - * / % ^ ! & | < > =  
:: ++ -- && || << >> ==  
+= -= *= /= %= ^= != &= |= <= >=  
&&= ||= <<= >>=  
-> =>  
// /*
```

3-Character Lookup Table

```
( ) [ ] { } ~ . , : ; + - * / % ^ ! & | < > =  
:: ++ -- && || << >> ==  
+= -= *= /= %= ^= != &= |= <= >=  
&&= ||= <<= >>=  
-> =>  
// /*
```

- Max punctuation length is 3
- 256 * 256 * 256 * 1 byte per entry
- = 16 MiB in total
- Larger than L3 cache

Exploit Token Shapes

```
( ) [ ] { } ~ . , : ; + - * / % ^ ! & | < > =  
:: ++ -- && || << >> ==  
+= -= *= /= %= ^= != &= |= <= >=  
&&= ||= <<= >>=  
-> =>  
// /*
```

- Punctuations can be:
 - A single character
 - A character repeated twice
 - A character followed by '='
 - A character repeated twice followed by '='
 - A character followed by '>'
 - /*

Implementation

```
1  for (;;) {
2      sourcePosition = skipWhitespace(sourcePosition);
3
4      bool repeat = sourcePosition[1] == sourcePosition[0];
5      int equalTestOffset = repeat ? 2 : 1;
6      bool equal = sourcePosition[equalTestOffset] == '=';
7      char extraTestCharacter = sourcePosition[0] == '/' ? '*' : '>';
8      bool extra = sourcePosition[1] == extraTestCharacter;
9
10     auto [tok, advance] = lookup(sourcePosition[0],
11                                 repeat, equal, extra);
12
13     switch (tok) {
14     case LINE_COMMENT_PLACEHOLDER:
15         sourcePosition = skipToEndOfLine(sourcePosition + 2);
16         output.recordComment(CommentKind::Line, /* source-information */);
17         continue;
18     case Token::StringLiteral:
19         sourcePosition = skipStringLiteral(sourcePosition);
20         sourcePosition += 1;
21         break;
22     ...
```

Implementation

```
1 struct Entry {
2     Token token : 6;
3     uint8_t advance : 2;
4 };
5
6 Entry lookup(char character, bool repeat, bool equal, bool extra) {
7     size_t index = (size_t)character
8                   | ((size_t)repeat << 7)
9                   | ((size_t)equal << 8)
10                  | ((size_t)extra << 9);
11     return table[index];
12 }
```

```
4     bool repeat = sourcePosition[1] == sourcePosition[0];
5     int equalTestOffset = repeat ? 2 : 1;
6     bool equal = sourcePosition[equalTestOffset] == '=';
7     char extraTestCharacter = sourcePosition[0] == '/' ? '*' : '>';
8     bool extra = sourcePosition[1] == extraTestCharacter;
9
10     auto [tok, advance] = lookup(sourcePosition[0],
11                                  repeat, equal, extra);
```

Pattern Table Summary

- 10 switch cases
- Table size is $128 * 2 * 2 * 2 = 1024$
- Lots of instructions
- Dependence between instructions

Pattern Table Benchmark

- 12.7M vs 15.5M lines/sec
- 281 vs 220 cycles/line
- Fewer branches (- 280k / - 7.9%)
- Fewer misses (- 9.9k / - 5.6%)
- Slightly higher miss rate (+ 0.12%)

On Randomized Tokens

- 35% vs 42% drop in throughput
- 65% vs 91% increase in cycles
- Branch miss rate 14% vs 15.2% (- 1.2%)
- Real source code is not random
 - Syntax
 - Conventions
 - Readability
 - ...

Expression Language

- Consider grammar consisting of only expressions

```
now = startTime + index(slide) * avgSlideDur;
```

```
avg[i] = (data[i] + data[i + 1]) / 2;
```

We have:

- Binary, unary, postfix operators
- Call, index expressions
- Parentheses
- Member access

But not:

- If statements
- Loops
- Functions
- Declarations

Expression Language

Two parsing states
when ignoring precedence

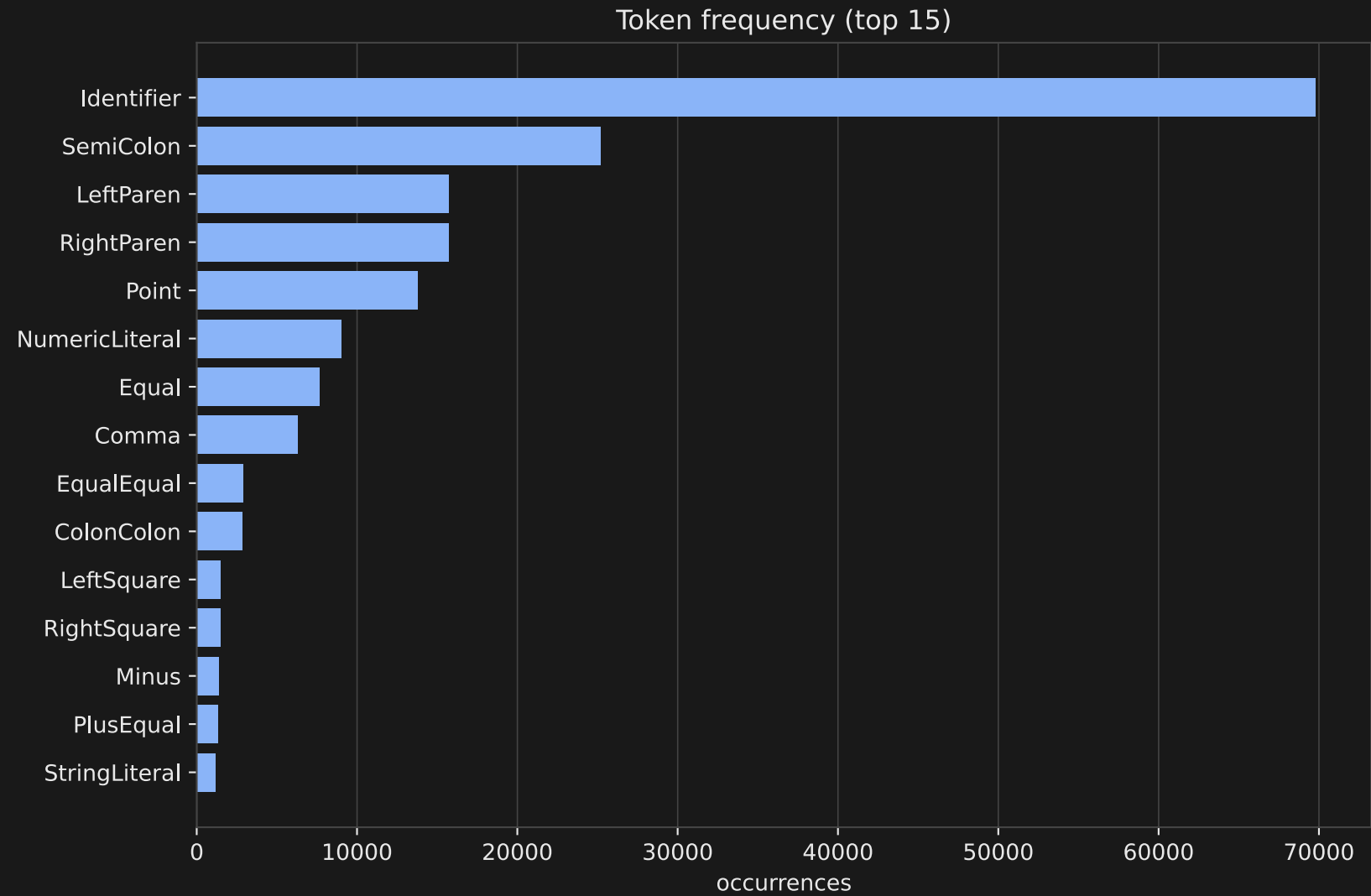
Expression:

- Identifiers, Literals
- Parentheses
- Unary operators

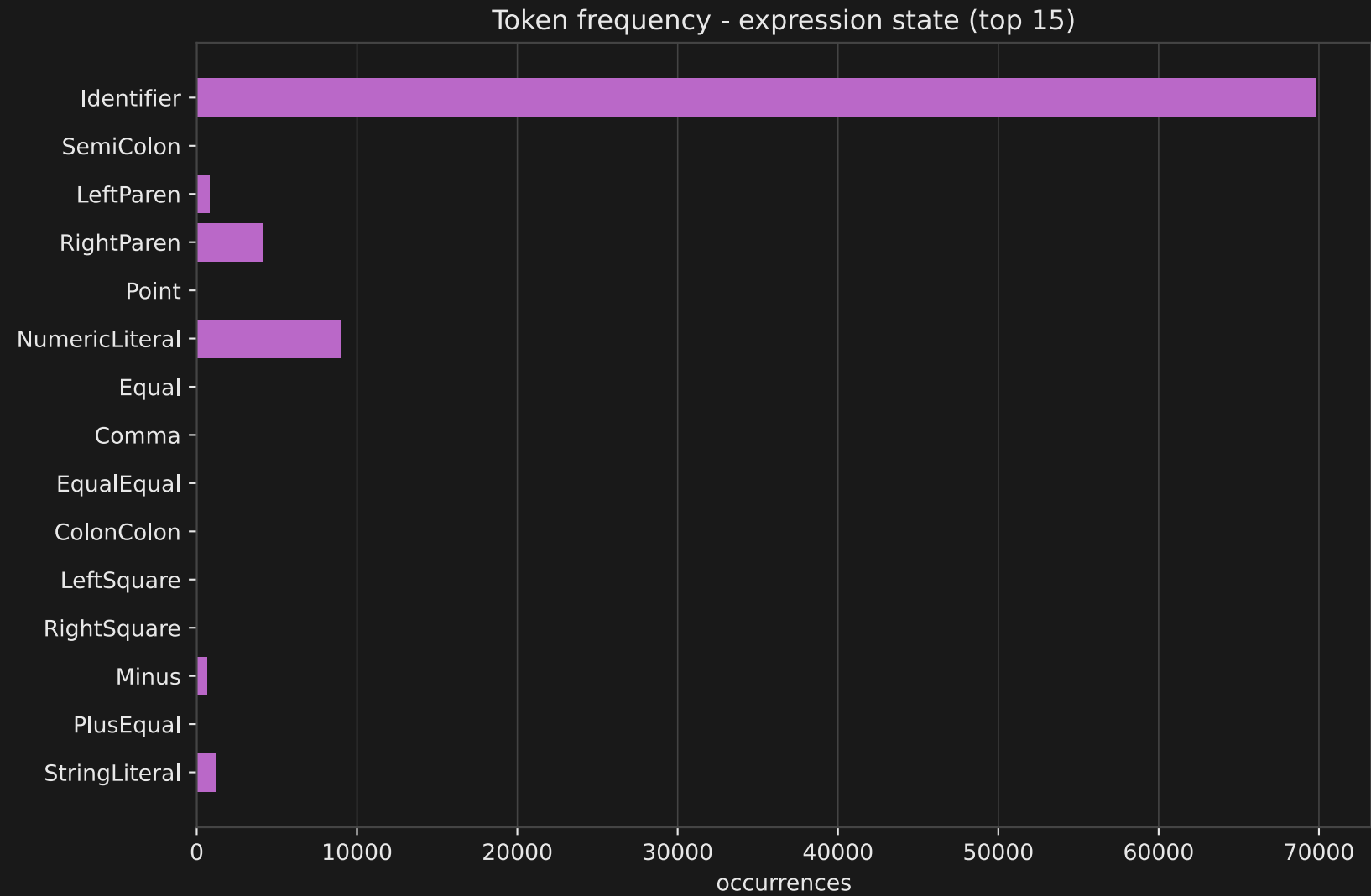
After expression:

- Binary operators
- Postfix operators
- Calls

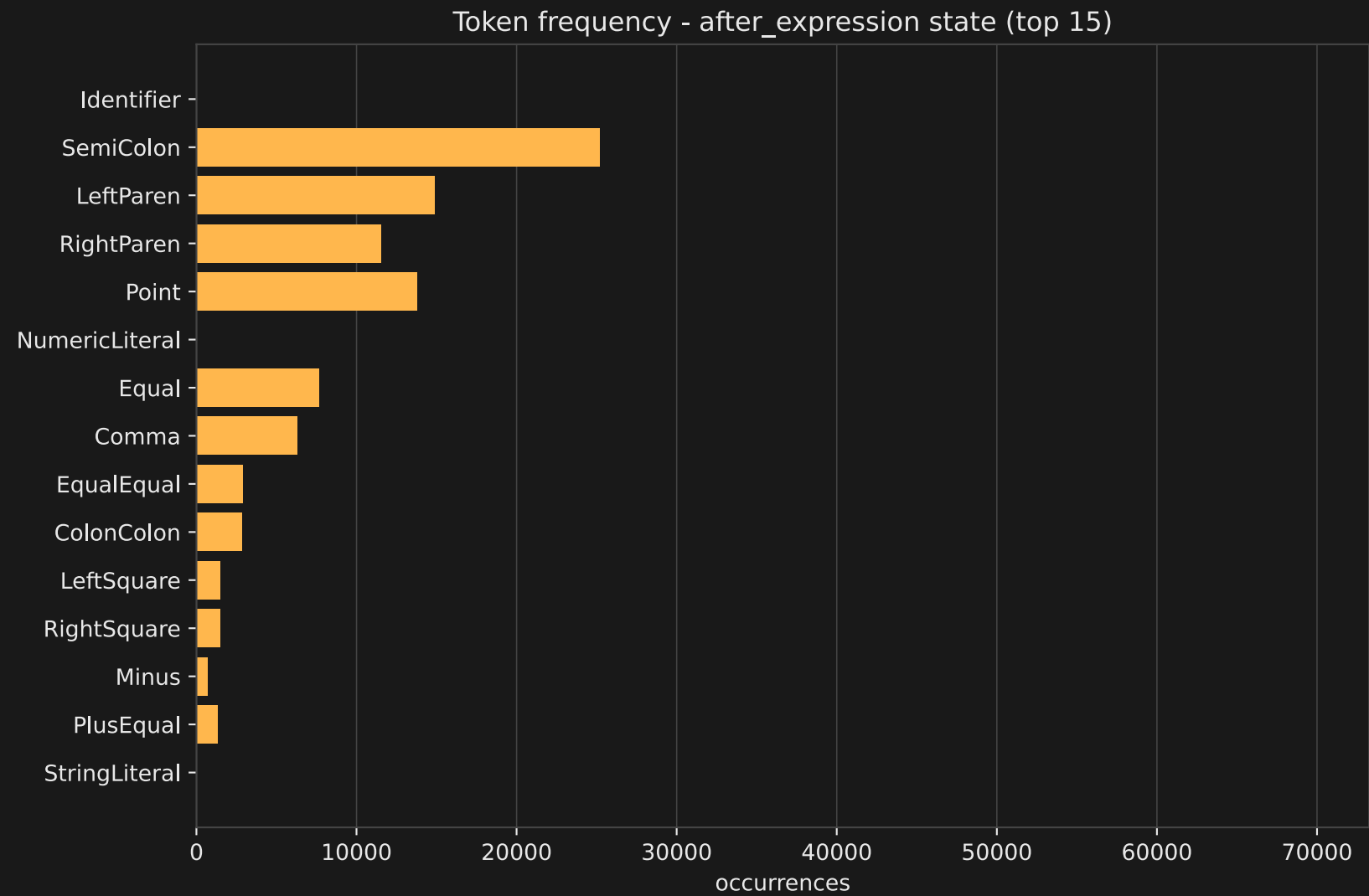
Token Frequency



Token Frequency



Token Frequency



Implementation

```
1  for (;;) {  
2      sourcePosition = skipWhitespace(sourcePosition);  
3      Token tok = Token::Invalid;  
4      switch (sourcePosition[0]) {  
5          case ',':  
6              tok = Token::Comma;  
7              sourcePosition += 1;  
8              break;  
9          ...  
10     }  
11     output.emitToken(tok, /* source-information */);  
12 }
```

Implementation

```
1 Token tok = Token::Invalid;
2 goto loop$no_emit;
3 {
4 loop$with_emit:
5     output.emitToken(tok, /* source-information */);
6 loop$no_emit:
7     sourcePosition = skipWhitespace(sourcePosition);
8     switch (sourcePosition[0]) {
9     case ',':
10         tok = Token::Comma;
11         sourcePosition += 1;
12         goto loop$with_emit;
13     ...
14 }
15 }
```

Implementation

```
1 Token tok = Token::Invalid;
2 goto expression$no_emit;
3 {
4 expression$with_emit:
5     output.emitToken(tok, /* source-information */);
6 expression$no_emit:
7     sourcePosition = skipWhitespace(sourcePosition);
8     switch (sourcePosition[0]) {
9         case ',':
10             tok = Token::Comma;
11             sourcePosition += 1;
12             goto error;
13     ...
14     }
15
16 after_expression$with_emit:
17     output.emitToken(tok, /* source-information */);
18 after_expression$no_emit:
19     sourcePosition = skipWhitespace(sourcePosition);
20     switch (sourcePosition[0]) {
21         case ',':
22             tok = Token::Comma;
```

+ Tokens

```
1 // expression
2 case '+':
3     if (sourcePosition[1] == '+') {
4         tok = Token::PlusPlus;
5         sourcePosition += 2;
6         goto expression$with_emit; // ++a
7     } else if (sourcePosition[1] == '=') {
8         tok = Token::PlusEqual;
9         sourcePosition += 2;
10        goto error; // += a
11    } else {
12        tok = Token::Plus;
13        sourcePosition += 1;
14        goto expression$with_emit; // +a
15    }
16
17 // after_expression
18 case '+':
19     if (sourcePosition[1] == '+') {
20         tok = Token::PlusPlus;
21         sourcePosition += 2;
22         goto after_expression$with_emit; // a++
```

/ Tokens And Comments

```
1 // expression
2 case '/':
3     if (sourcePosition[1] == '*') {
4         sourcePosition = skipBlockComment(sourcePosition + 2);
5         sourcePosition += 2;
6         output.recordComment(CommentKind::Block, /* source-information */);
7         goto expression$no_emit;
8     } else if (sourcePosition[1] == '/') {
9         sourcePosition = skipToEndOfLine(sourcePosition + 2);
10        output.recordComment(CommentKind::Line, /* source-information */);
11        goto expression$no_emit;
12    } else if (sourcePosition[1] == '=') {
13        tok = Token::SlashEqual;
14        sourcePosition += 2;
15        goto error; // /= a
16    } else {
17        tok = Token::Slash;
18        sourcePosition += 1;
19        goto error; // / a
20    }
21
22 // after expression
```

2-State Summary

- Two copies of switch and branch
- Differ only in goto targets
- Better branch prediction
- Theoretically same instructions
 - Cannot be slower?

2-State Benchmark

- Different benchmark data
- 16.2M vs 17.1M lines/sec
- 1-State identical to Switch And Branch
- More instructions (+ 1.36M / + 11.4%)
- More branches (+ 0.10M / + 4.9%)
- Fewer misses (- 4.0k / - 3.4%)
- Lower miss rate (- 0.44%)

What Went Wrong

- Breaks conditional moves
- Larger function
 - Worse codegen decisions
 - More register pressure

Why Continue In This Direction

- Can we validate everything?
- What happens for a real grammar?
- Applicable grammars?

Validate Everything

- ' , ' is only valid in some contexts
- Need to match ' (' and ') '
- Solution: Add a scope stack

Validate Everything

```
1 // after_expression
2 case '(':
3     tok = Token::LeftParen;
4     sourcePosition += 1;
5     goto expression$with_emit;
6 case ',':
7     tok = Token::Comma;
8     sourcePosition += 1;
9     goto expression$with_emit;
10 case ')':
11     tok = Token::RightParen;
12     sourcePosition += 1;
13     goto after_expression$with_emit;
```

Validate Everything

```
1 // after_expression
2 case '(':
3     scopeStack.push(Scope::Call);
4     tok = Token::LeftParen;
5     sourcePosition += 1;
6     goto expression$with_emit;
7 case ',':
8     scopeStack.test(Scope::Call);
9     tok = Token::Comma;
10    sourcePosition += 1;
11    goto expression$with_emit;
12 case ')':
13    scopeStack.pop(Scope::Call);
14    tok = Token::RightParen;
15    sourcePosition += 1;
16    goto after_expression$with_emit;
```

Real Grammar

- Needs to be predictable: Template or comparison?
 - Rust: Inferred or Turbofish

```
let a: Vec<i32> = Vec::new();  
let b = Vec::<i32>::new();
```

- Carbon: Explicit () or implicit []

```
fn Sort[T: Comparable](ref v: Vector(T));
```

- Cpp2: Whitespace sensitive

```
f<int>(c);    // template  
a<b;         // comparison  
a <b>(c);     // comparison chain -> error
```

- Charge: Braces

```
let a: vector{int} = ();  
let b = vector{int}();
```

Real Grammar

- Need an abstraction

```
1 SwitchState expression() then error
2     punctuation +
3         emitToken PlusExpr
4         next expression
5     punctuation ++
6         emitToken PreIncrementExpr
7         next expression
8
9     identifier
10         emitToken IdentifierExpr
11         next after_expression
```

Walkthrough

```
struct Foo: {  
    bar: int;  
}
```

Accept!

Simple Output Benchmark

- 12.8M vs 15.5M lines/sec
- 280 vs 220 cycles/line
- More instructions (+ 8.73M / + 42.0%)
- More branches (+ 1.74M / + 49.3%)
- Fewer misses (- 18.8k / - 10.7%)
- Lower miss rate (- 2.0%)
- Does validate syntax

No Output Benchmark

- 19.2M vs 15.5M lines/sec
- 225 vs 220 cycles/line

Advantages

- Outputs validated token stream
 - Ideal for formatting
- Error recovery can be automated
- States can be hooked
 - Count '+' unary vs binary usage
 - Build declaration lookup tables
 - Aggregate designated call arguments
- Dense memory layout
 - Alternative: Carbon's post-order layout

Disadvantages

- Requires tight integration
 - Can't directly replace existing parsers
 - Requires customization to be useful
- High initial implementation burden
 - 1500 lines of Python
 - 930 lines of state machine
 - Needs a library

Sema Benchmark

- Builds identifier and lookup tables
- 7.2M lines/sec
- 445 cycles/line

Parser Or Lexer

- Pro:
 - Validates the grammar
 - Tokens are context sensitive
- Con:
 - Outputs (renamed) lexer tokens
 - Does not resolve precedence

Conclusions

- Did not succeed in writing a faster lexer
 - Improvements could be possible
- Instead separate token recognition per parse state
 - Improves predictability
 - Slower but can do more
- Perfect for syntax validation and formatting
- Great for parsing if designed around

Carbon Benchmark

	lines/ s	cyc/ line	instr	branches	br- miss	miss%
Charge Sema	7.2M	445	45.82M	7.27M	188.3k	2.59%
Carbon Lex	6.1M	689	71.22M	11.52M	288.4k	2.50%
Carbon Lex + Parse	3.0M	1379	158.3M	25.08M	437.3k	1.74%

Thank you!

Torben Thaysen

github.com/Torbenx/charge

Branch: parsertalk

Bonus

Benchmark Scenarios

Benchmark Scenarios

- Compiler → start process, parse, kill process
 - Single cold run
- Language server → reparse on each key press
 - Repeated hot runs
- Throughput on infinite file
 - Hot runs on very large file

Benchmark Fixed Repetitions

- Measure branch miss rate vs repetitions
- Not supported by Google Benchmark
- [Blog post by Pramod Kumbhar](#)
- Use `perf stat --control`

Bash Wrapper

```
1 # create two fifos
2 mkfifo "perf_fd.ctl"
3 mkfifo "perf_fd.ack"
4
5 # associate file descriptors
6 exec {perf_ctl_fd}<>"perf_fd.ctl"
7 exec {perf_ack_fd}<>"perf_fd.ack"
8
9 # fifos can be unlinked immediately
10 unlink "perf_fd.ctl"
11 unlink "perf_fd.ack"
12
13 # set env vars for application
14 export PERF_CTL_FD=${perf_ctl_fd}
15 export PERF_ACK_FD=${perf_ack_fd}
16
17 # start perf with the associated file descriptors
18 perf stat \
19     --event=cycles:u,instructions:u,branches:u,branch-misses:u \
20     --delay=-1 --repeat=100 --log-fd=1 \
21     --control fd:${perf_ctl_fd},${perf_ack_fd} \
22     -- ./build/charge "$@" 2> /dev/null
```

C++ Enabler

```
1 struct PerfEnable {
2     PerfEnable() {
3         char* ctlFdStr = std::getenv("PERF_CTL_FD");
4         char* ackFdStr = std::getenv("PERF_ACK_FD");
5
6         write(ctlFd, "enable", 7);
7         std::array<char, 5> result = {}; // Replies "ack\n\0"
8         int bytesRead = read(ackFd, result.data(), 5);
9         assert(bytesRead == 5);
10    }
11
12    ~PerfEnable() {
13        write(ctlFd, "disable", 8);
14        std::array<char, 5> result = {}; // Replies "ack\n\0"
15        int bytesRead = read(ackFd, result.data(), 5);
16        assert(bytesRead == 5);
17    }
18
19    int ctlFd = 0;
20    int ackFd = 0;
21 };
```

C++ Enabler

```
1 {  
2     PerfEnable enable;  
3     for (int i = 0; i < repeats; i++) {  
4         LexerOutput output { source };  
5         const char* end = lexSwitchAndBranch(source.data(), output);  
6         assert(end == source.data() + source.size());  
7     }  
8 }
```

